

Next.js Middleware allows you to run code before a request completes, enabling you to modify responses, rewrite URLs, handle authentication, or manage headers and cookies. Middleware executes globally across your application before cached content and route matching.  Next.js +1

1. Creating Basic Middleware

Create a `middleware.ts` (or `.js`) file in the **root** of your project (same level as `app` or `pages`, or inside `src` if applicable).  Next.js +1

typescript

```
// middleware.ts
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function middleware(request: NextRequest) {
  // Example: Redirect all users from /old-page to /new-page
  if (request.nextUrl.pathname.startsWith('/old-page')) {
    return NextResponse.redirect(new URL('/new-page', request.url))
  }

  // Allow the request to proceed
  return NextResponse.next()
}

// Optional: Configure a matcher to filter specific paths
export const config = {
  matcher: '/((?!api|_next/static|_next/image|favicon.ico).*)',
}
```

2. Using `createMiddleware` from `next-intl`

When working with internationalization (i18n), specifically the `next-intl` library, `createMiddleware` is used to manage locale-based routing.  next-intl +1

typescript

```
// middleware.ts
import createMiddleware from 'next-intl/middleware';

// 1. Configure the middleware
export default createMiddleware({
  // A list of all locales that are supported
  locales: ['en', 'de'],
```

```
// Used when no Locale matches
defaultLocale: 'en'
});

export const config = {
  // Match only internationalized pathnames
  matcher: ['/', '/(de|en)/:path*']
};
```

3. Key Concepts

- **Request/Response Handling:** You use `NextRequest` and `NextResponse` to inspect and manipulate incoming requests and outgoing responses.
- **Matchers:** Use `config.matcher` to restrict middleware to specific paths, which is crucial for performance and ensuring middleware doesn't run on static files or API routes.
- **Order of Execution:** Next.js runs middleware before the route matches. Only one `middleware.ts` file is supported per project.  Next.js +4

4. Common Use Cases

- **Authentication:** Protecting routes by checking cookies/tokens, as described on the [Clerk blog](#).
- **Redirects & Rewrites:** Handling URL redirects (e.g., `/home` to `/dashboard`).
- **Internationalization (i18n):** Handling language detection and routing via `next-intl`.  next-intl +4

Note: For complex scenarios where multiple libraries need middleware (e.g., `Auth0` + `next-intl`), you may need to manually compose them, as discussed in [NextJS with Auth0 and Next-Intl on Auth0 Community](#) and the [next-intl GitHub discussions](#).